

* Data structures (Multivalued variable)

⇒ Python provides several built-in data structures that help us organizing and manage data efficiently.

Different types of DS are:

1. List 2. Tuple 3. Set 4. Dict 5. Str

(i) List

⇒ It's is multivalued heterogenous data collectors.

- It is ordered, mutable and it will allow duplicates its represented by `[]`.

Ex: `l1 = [15, "Kiran", 2004, 21.5, True]`
`l1`

o/p `[15, 'Kiran', 2004, 21.5, True]`

* `l1.append(45)`

`l1`

o/p `[15, 'Kiran', 2004, 21.5, True, 45]`

* ll.insert(4, False)

ll

Q12 [15, 'Kiran', 2004, 21.5, False, True, 45]

* ll.pop()

ll

Q12 [15, 'Kiran', 2004, 21.5, False, True]

* ll.pop(3)

ll

Q12 [15, 'Kiran', 2004, False, True]

* ll.remove(False)

ll

Q12 [15, 'Kiran', 2004, True]

We displaying in the sequence

for i in ll:

print(i)

Q12 15

Kiran

2004

True

we adding 2 list of the individual elements sum

$l2 = [2, 5, 67, 89, 23, 12]$

$l3 = [76, 54, 23, 89, 23, 80]$

$sl = []$

for i in range(0, 6, 1):

$s = l2[i] + l3[i]$

$sl.append(s)$

print(sl)

Q12 $[78, 59, 90, 178, 46, 92]$

Ex: create even, odd, prime number list from 1 to 20 numbers

$en = []$

$on = []$

$pn = []$

for i in range(1, 21, 1):

if $(i \% 2 == 0)$:

$en.append(i)$

else:

$on.append(i)$

if $(i != 1)$:


```

for j in range(2, i):
    if C[i-j] == 0:
        break
    else:
        pn.append(C[i-j])

```

print(Cn)

print(Con)

print(Pn)

Q [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
[2, 3, 5, 7, 11, 13, 17, 19]

Ex 1: a = [1, 2, 3, 4, 5]
b = [4, 5, 6, 7, 8]
c = a
for i in b:
 if i not in a:
 c.append(i)

print(c)

Q [1, 2, 3, 4, 5, 6, 7, 8]

Ex 1: a = [1, 2, 3, 4, 5]
b = [4, 5, 6, 7, 8]
c = []
for i in a:

```
if i not in b:  
    c.append(i)
```

```
for j in b:  
    if j not in a:  
        c.append(j)
```

```
print(c)
```

Q12 [1, 2, 3, 6, 7, 8]

* Slicing and list comprehension

slicing [start: end: inc/dec]

- default start is 0
- default end position is upto last position
- default 'inc/dec' is 1

Ex: list1 [15, "Kiran", 87.9, True, 35, "Python", 88]

list1

Q12 [15, 'Kiran', 87.9, True, 35, 'Python', 88]

- list1[0:3]

Q12 [15, 'Kiran', 87.9]

• Extract last 3 elements

`list1[-3:]`

o/p `[35, 'Python', 88]`

• Extract even indexed elements

`list1[1:7:2]`

o/p `[15, 87.9, 35, 88]`

• Extract elements whose index is multiple of 3

`list1[3::3]`

o/p `[True, 88]`

• Reverse the list

`list1[::-1]`

`[88, 'Python', 35, True, 87.9, 'Kiran', 15]`